Triggers

- Definición y casos de uso
- Arquitectura
  - 4 pilares
  - Errores
  - OLD/NEW
- Ejercicios

Triggers

Un **Trigger** (o disparador) es un código SQL procedimental que el motor de la base de datos ejecuta de forma **automática e implícita** cada vez que ocurre un evento específico en una tabla.

Su función es actuar como un **policía de seguridad** que puede interceptar una operación, cancelarla y **lanzar un error explícito**.

¿Para qué se utilizan?

- Auditoría y Trazabilidad de Cambios
- Validaciones Complejas y Reglas de Negocio
- Normalización y Limpieza Automática de Datos
- Mantenimiento de Datos Derivados y Contadores
- Bloqueo de Borrado Físico / Forzado de Borrado Lógico

Triggers
Arquitectura

La estructura de un Trigger se define mediante cuatro componentes fundamentales:

- **Tabla Asociada:** pertenece estrictamente a una sola tabla de la base de datos.
- **Evento Disparador:** La operación de datos que lo activa (INSERT, UPDATE o DELETE).
- **Momento de Ejecución :**
  - **BEFORE:** Se ejecuta *antes* de que el cambio se aplique físicamente en la tabla. Ideal para validar o transformar datos entrantes.
  - **AFTER:** Se ejecuta *después* de que la modificación se aplicó en la tabla. Ideal para auditorías, trazabilidad y acciones en cascada.
- Los triggers se ejecutan nivel de fila mediante la cláusula **FOR EACH ROW**

```
DELIMITER //
CREATE TRIGGER trg_film_valida_duracion
BEFORE INSERT ON film
FOR EACH ROW
BEGIN
    -- Si se especifica una duración, validamos que no sea 0 o negativa
    IF NEW.length IS NOT NULL AND NEW.length <= 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Error: La duración de la película debe ser mayor a 0 minutos.';
    END IF;
END //
DELIMITER ;
```

Triggers

Arquitectura Error

- **SIGNAL**: Es el comando que le ordena al motor de la base de datos disparar una excepción en ese preciso instante.
- **SQLSTATE**: Identificador estándar de 5 caracteres que clasifica la naturaleza del error según la norma ANSI SQL.
- **'45000'**: Es el código universal reservado para **excepciones personalizadas definidas por el usuario** (*User-Defined Exception*).

El estándar SQL categoriza los tipos de fallos según sus dos primeros dígitos.

```
DELIMITER //
CREATE TRIGGER trg_film_valida_duracion
BEFORE INSERT ON film
FOR EACH ROW
BEGIN
    -- Si se especifica una duración, validamos que no sea 0 o negativa
    IF NEW.length IS NOT NULL AND NEW.length <= 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Error: La duración de la película debe ser mayor a 0 minutos.';
    END IF;
END //
DELIMITER ;
```

Ejercicios

1. Crear un trigger sobre la tabla payment que impida registrar cobros con un monto menor o igual a 0.

2. Crear un trigger sobre la tabla customer que verifique que el correo electrónico ingresado contenga el símbolo @. Si no lo contiene, debe abortar la operación.

3. Crear un trigger sobre la tabla rental que impida registrar un alquiler donde la fecha de devolución sea anterior a la fecha de inicio.

4. Crear un trigger sobre la tabla staff que valide que el nombre de usuario tenga una longitud mínima de 4 caracteres.

En todos los casos se deberán contemplar mensajes de error.

Triggers

Arquitectura – OLD/NEW

Para poder procesar la información, el motor le otorga al Trigger acceso a los datos antes y después del cambio mediante dos registros especiales OLD y NEW

OLD y NEW son registros temporales que el motor de la base de datos crea en memoria exclusivamente durante la ejecución de un Trigger FOR EACH ROW para representar el estado de la fila afectada

```
1   DELIMITER //
2   CREATE TRIGGER trg_customer_email_lower
3   BEFORE INSERT ON customer
4   FOR EACH ROW
5   BEGIN
6       -- Modificamos el valor entrante usando la variable NEW
7       SET NEW.email = LOWER(NEW.email);
8   END //
9   DELIMITER ;
```

Evento	Registro OLD (Estado Previo)	Registro NEW (Estado Nuevo)
INSERT	NULL (No existía previamente)	Contiene los datos ingresados
UPDATE	Contiene los datos originales antes del cambio	Contiene los nuevos datos a guardar
DELETE	Contiene los datos de la fila eliminada	NULL (La fila deja de existir)

EJERCICIOS

1. Crear una tabla de historial y un trigger que guarde el correo antiguo (OLD) y el nuevo correo (NEW) cada vez que un cliente modifique su dirección de email en la tabla
2. Registrar la variación del precio de alquiler (rental_rate) de las películas. Debe guardar la tarifa previa, la nueva tarifa, la diferencia calculada y la fecha de cambio.
3. Crear un historial de respaldo cuando un registro sea eliminado de la tabla *rental*
4. Auditar los cambios de sucursal de los empleados en Sakila. Cada vez que el campo store_id sea modificado, el Trigger debe registrar la tienda de origen y la tienda de destino

```
1   DELIMITER //
2   CREATE TRIGGER trg_customer_email_lower
3   BEFORE INSERT ON customer
4   FOR EACH ROW
5   BEGIN
6       -- Modificamos el valor entrante usando la variable NEW
7       SET NEW.email = LOWER(NEW.email);
8   END //
9   DELIMITER ;
```